

TP Redis

Cours + Pratique

De zéro à un projet fil rouge CityFlow, avec redis-commander comme interface

Module	NoSQL — B3 INFO Tronc Commun
Séance	Séance 3 (Jour 2 matin)
Durée estimée	4 à 5 heures (cours + TP)
Outils requis	Docker Desktop, navigateur web, éditeur de texte
Public	Étudiants B3 INFO (INFRES + CYBSEC)
Pré-requis	TP MongoDB terminé, notions Docker, JavaScript de base
Formateur	KAKE Abdoulaye
Année	2025 / 2026

Comment utiliser ce document ? Ce TP combine du cours et de la pratique en alternance. Les passages « COURS » présentent les concepts. Les passages « VOTRE TOUR » sont des exercices à faire individuellement ou en binôme. Les passages « CityFlow » alimentent le projet fil rouge. Avancez à votre rythme, ne sautez aucune étape.

Redis : philosophie et positionnement

L'origine : un side project devenu standard

Redis (**RE**mote **D**ictionary **S**erver) a été créé en 2009 par **Salvatore Sanfilippo**, alias « antirez », développeur italien. À l'origine, il voulait améliorer la scalabilité d'un service de statistiques web et a écrit Redis en quelques jours pour résoudre son propre problème. Il l'a open-sourcé sans imaginer son destin : Redis est devenu en moins de 10 ans la base clé-valeur la plus utilisée au monde.

*Depuis 2020, Salvatore s'est retiré du développement actif, et Redis est désormais maintenu par Redis Inc. (anciennement Redis Labs). En mars 2024, Redis Inc. a changé la licence pour SSPL/RSAL, ce qui a entraîné un fork communautaire nommé **Valkey**, soutenu par AWS, Google et Oracle. Pour ce cours, nous utilisons Redis 7 (la dernière version sous licence BSD), mais Valkey est totalement compatible.*

Caractéristiques essentielles

Modèle	Clé-valeur avec structures de données riches (String, Hash, List, Set, Sorted Set, Stream, etc.)
Position CAP	CP en mode standalone, AP avec Redis Cluster
Mono-thread	Un seul thread principal traite les commandes en séquence — paradoxalement c'est ce qui rend Redis simple et rapide
Atomicité	Chaque commande est atomique par construction. Transactions multi-commandes via MULTI/EXEC ou scripts Lua
Persistance	Optionnelle : RDB (snapshots), AOF (append-only log), ou les deux combinés
Réplication	Maître-esclave asynchrone, ou cluster avec sharding automatique (16384 hash slots)
Version actuelle	Redis 7.x (utilisée dans ce cours), Valkey 7.x équivalent

Quand utiliser Redis (et quand ne pas)

- **OUI** : Cache de résultats coûteux (requêtes SQL, appels API, calculs)
- **OUI** : Sessions utilisateurs avec expiration automatique
- **OUI** : Compteurs temps réel (vues, likes, rate limiting)
- **OUI** : Leaderboards et classements en temps réel
- **OUI** : File d'attente de jobs ou messages éphémères
- **OUI** : Pub/sub temps réel (notifications, chat)
- **NON** : Source de vérité critique pour données financières (sauf avec persistance maximale)
- **NON** : Volumes très importants qui dépasseraient la RAM disponible (sauf cluster)

- **NON** : Requêtes complexes ad-hoc sur des données (Redis n'est pas une base analytique)

Partie 1 — Mise en route avec Docker et redis-commander

1.1. Objectif de cette partie

À la fin de cette partie, vous aurez **Redis qui tourne dans Docker** et **redis-commander** accessible depuis votre navigateur. Vous ajouterez ces services à votre docker-compose.yml CityFlow déjà existant — chaque base s'empile à la précédente.

1.2. Pourquoi Redis ? Pourquoi redis-commander ?

Redis est une base de données **clé-valeur en mémoire**. Là où MongoDB stocke des documents structurés sur disque, Redis stocke des paires clé→valeur en RAM. Conséquence : des temps de réponse en dessous de la milliseconde, mais une volumétrie limitée à la mémoire disponible. C'est l'outil parfait pour le cache, les sessions, les compteurs temps réel et les classements.

redis-commander est une interface web open-source pour Redis. Comme mongo-express, elle tourne dans Docker et s'utilise via le navigateur. Vous allez **cliquer et explorer** les structures de données Redis sans avoir à apprendre redis-cli immédiatement.

Note importante : Redis est avant tout utilisé via du code applicatif (Node.js, Python, .NET) en production. redis-commander est un outil d'**administration et de debug**, comme phpMyAdmin l'est pour MySQL. Vous l'utilisez ici pour apprendre les concepts en visualisant.

1.3. Étape 1 : continuer avec votre dossier cityflow-nosql

Vous travaillez toujours dans le même dossier ./cityflow-nosql créé lors du TP MongoDB. Si MongoDB tourne encore (vous pouvez vérifier avec docker compose ps), c'est parfait : on ajoute juste deux services au stack existant. Si vous avez tout arrêté, ce n'est pas grave, on relancera tout à la fin.

```
cd ./cityflow-nosql
docker compose ps # vérifier l'état actuel
```

1.4. Étape 2 : ajouter Redis et redis-commander au docker-compose.yml

Ouvrez votre docker-compose.yml dans VSCode. À la suite du service mongo-express, **avant la section `volumes`**, ajoutez les deux nouveaux services :

```
redis:
  image: redis:7-alpine
  container_name: cityflow-redis
  restart: unless-stopped
  ports:
    - "6379:6379"
  volumes:
```

```
- redis_data:/data
command: redis-server --requirepass cityflow2025 --appendonly yes
networks:
- cityflow-net

redis-commander:
image: rediscommander/redis-commander:latest
container_name: cityflow-redis-commander
restart: unless-stopped
ports:
- "8082:8081"
environment:
  REDIS_HOSTS: local:redis:6379:0:cityflow2025
  HTTP_USER: student
  HTTP_PASSWORD: nosql2025
depends_on:
- redis
networks:
- cityflow-net
```

Puis modifiez la section volumes pour ajouter redis_data :

```
volumes:
  mongo_data:
  redis_data:
```

Comprendre la configuration Redis

- ``image: redis:7-alpine`` : image Redis 7 sur Alpine Linux (plus léger qu'Ubuntu)
- ``6379:6379`` : Redis écoute sur le port 6379 (standard)
- ``--requirepass cityflow2025`` : protège Redis par mot de passe
- ``--appendonly yes`` : active la persistance AOF (les données survivent au redémarrage)
- ``redis_data:/data`` : le volume Docker où Redis écrit ses fichiers de persistance

Comprendre la configuration redis-commander

- ``8082:8081`` : on expose redis-commander sur **8082** (8081 est déjà pris par mongo-express)
- ``REDIS_HOSTS: local:redis:6379:0:cityflow2025`` : format nom:host:port:db:password
- ``HTTP_USER / HTTP_PASSWORD`` : authentification basique de l'interface

⚠ **Conflit de port à éviter** : mongo-express utilise déjà le port 8081. C'est pourquoi on expose redis-commander sur 8082. Vous accéderez à mongo-express sur localhost:8081 et redis-commander sur localhost:8082. Pas de confusion !

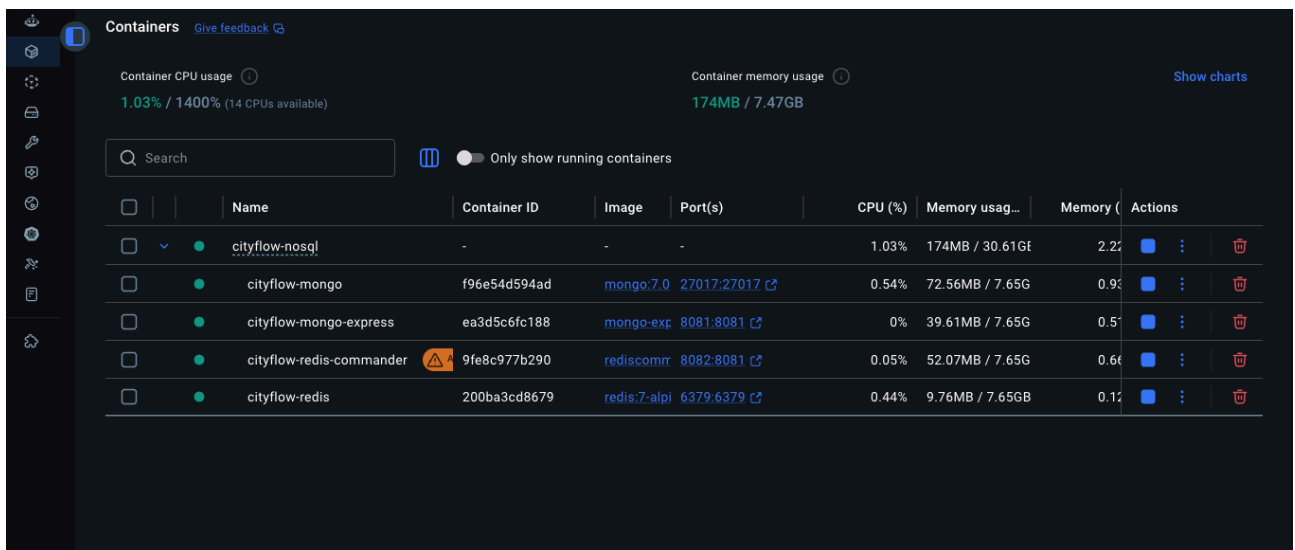
1.5. Étape 3 : lancer la nouvelle stack

```
# Relancer toute la stack (les services manquants sont ajoutés)
docker compose up -d

# Vérifier les 4 conteneurs
```

docker compose ps

Vous devez voir maintenant **4 conteneurs** à l'état Up : cityflow-mongo, cityflow-mongo-express, cityflow-redis, cityflow-redis-commander.



The screenshot shows the Docker Desktop interface. At the top, it displays 'Container CPU usage' at 1.03% / 1400% (14 CPUs available) and 'Container memory usage' at 174MB / 7.47GB. Below this is a search bar and a toggle for 'Only show running containers'. A table lists the running containers:

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage	Memory (MB)	Actions
☐	cityflow-nosql	-	-	-	1.03%	174MB / 30.61Gi	2.22	
☐	cityflow-mongo	f96e54d594ad	mongo:7.0	27017:27017	0.54%	72.56MB / 7.65G	0.93	
☐	cityflow-mongo-express	ea3d5c6fc188	mongo-exp	8081:8081	0%	39.61MB / 7.65G	0.51	
☐	cityflow-redis-commander	9fe8c977b290	rediscomm	8082:8081	0.05%	52.07MB / 7.65G	0.64	
☐	cityflow-redis	200ba3cd8679	redis:7-alpi	6379:6379	0.44%	9.76MB / 7.65GB	0.12	

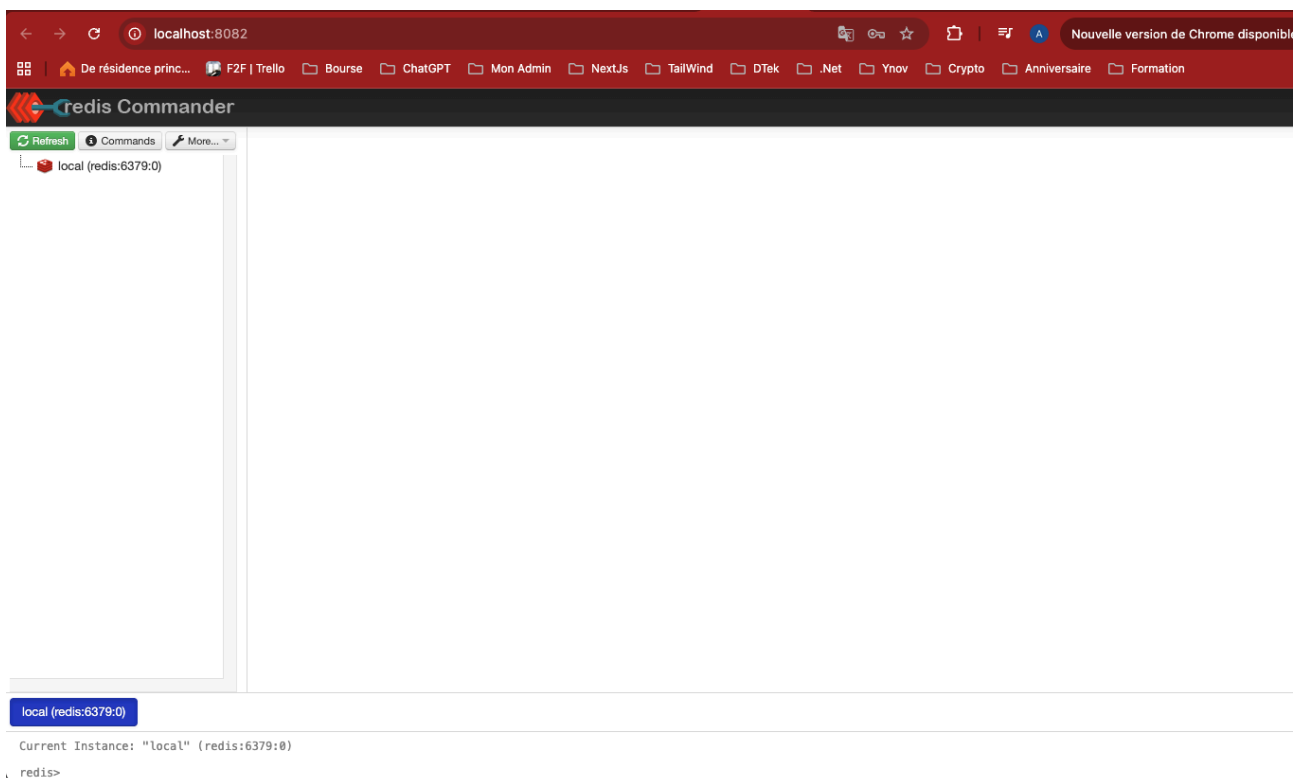
1.6. Étape 4 : ouvrir redis-commander

Dans votre navigateur, ouvrez : <http://localhost:8082>

Authentifiez-vous :

- **Utilisateur** : student
- **Mot de passe** : nosql2025

Vous arrivez sur l'interface redis-commander. Vous voyez à gauche la connexion local avec 0 clés (notre base est vide). C'est normal et c'est ce qu'on va remplir.



VOTRE TOUR — Validation de la mise en route

1. docker compose ps montre **4 conteneurs Up** (Mongo, mongo-express, Redis, redis-commander).
2. <http://localhost:8081> (mongo-express) marche toujours et montre votre base tp_mongo.
3. <http://localhost:8082> (redis-commander) ouvre l'interface après authentification.
4. À gauche, vous voyez la connexion local avec « 0 keys ».

Partie 2 — Découverte de l'interface redis-commander

2.1. Anatomie de l'interface

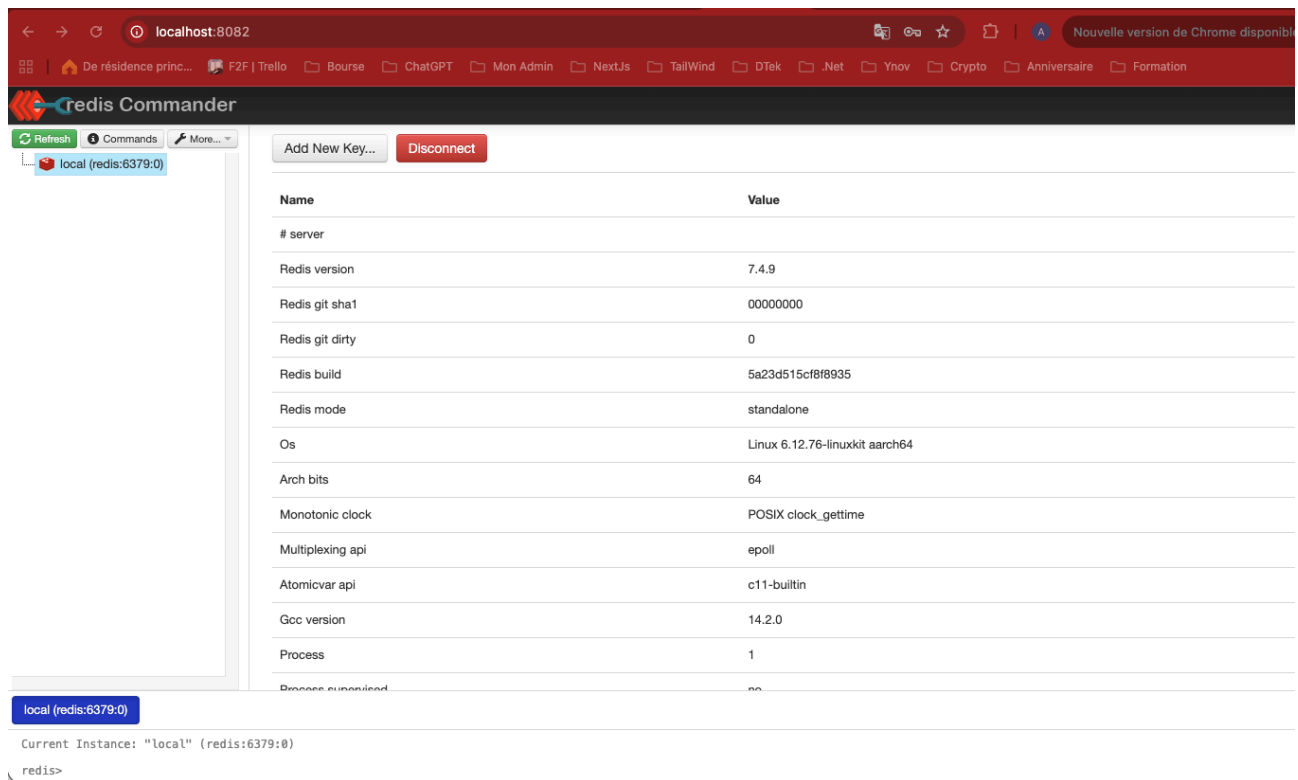
Vue générale

- **Panneau de gauche** : arborescence des bases Redis
- **Barre du haut** : champ pour exécuter directement des commandes Redis (Console)
- **Zone centrale** : vue détaillée de la clé sélectionnée (valeur, TTL, type)
- **Boutons d'action** : Add Key, Refresh, Disconnect, etc.

Particularité de Redis : pas de tables ni collections

Contrairement à MongoDB qui organise les documents en **collections**, Redis stocke ses clés **dans un espace global plat**. Toutes les clés cohabitent au même niveau. Pour structurer, on utilise des **conventions de nommage** avec le séparateur : (par exemple user:42:name, session:abc123, leaderboard:weekly).

redis-commander affiche ces clés en arborescence virtuelle basée sur le séparateur :, ce qui donne l'illusion de dossiers. Vous le verrez quand on aura inséré des clés.



VOTRE TOUR — Exploration libre (10 min)

Sans rien modifier, explorez l'interface pendant 10 minutes :

1. Cliquez sur la connexion local à gauche pour voir l'arborescence.
2. Cliquez sur le bouton **Console** en haut → vous voyez un terminal Redis. Tapez PING puis Entrée. Réponse attendue : PONG.
3. Tapez INFO server dans la console pour voir la version de Redis.
4. Repérez les boutons : **Add Key** (en haut à gauche), **Refresh**.
5. Notez sur papier les 3 fonctionnalités qui vous intriguent le plus.

Aucune création de clé pendant cette exploration.

Partie 3 — Cours : philosophie de Redis et structures

3.1. Le pari fondateur de Redis : tout en mémoire

Redis (REmote DIctionary Server) lors de sa création, Ils ont fait un choix radical pour l'époque : **stocker toutes les données en mémoire RAM**, et seulement secondairement les persister sur disque. Cette décision semble dangereuse — la RAM se perd au redémarrage — mais elle donne à Redis sa vitesse incomparable.

Niveau de stockage	Latence typique	Pour comparaison
Registre CPU	~1 ns	Référence absolue
Cache CPU L1/L2	~5 ns	Très rapide mais minuscule
RAM (Redis)	~100 ns	Redis exploite ce niveau
SSD NVMe	~100 µs	1000× plus lent que la RAM
SSD classique	~500 µs	5000× plus lent
Disque HDD	~10 ms	100 000× plus lent
Réseau internet	~50-500 ms	(comparaison)

Conséquence pratique : une commande Redis simple (GET, SET) prend **moins d'une milliseconde**, là où une requête SQL classique prend 10-100 ms. Sur une page web qui fait 50 requêtes BDD, c'est la différence entre une page qui charge en 100ms et une qui rame.

3.2. Les 5 structures de données principales

Redis n'est pas un simple clé → valeur plat. C'est ce qui le distingue de Memcached ou d'un cache HashMap basique : Redis offre **5 structures de données spécialisées**, optimisées pour des cas d'usage précis. Maîtriser ces 5 structures = savoir résoudre 90% des problèmes.

Structure 1 — String : la base

Le type le plus simple : une clé associée à une valeur (texte, nombre, JSON sérialisé, données binaires...). Malgré sa simplicité, c'est la structure la plus utilisée parce qu'elle couvre les cas de cache simple, compteurs, et flags.

```
# Stocker une valeur simple
SET user:42:name "Alice"
GET user:42:name → "Alice"

# Stocker un JSON sérialisé
SET user:42 '{"name":"Alice","age":28,"city":"Lyon"}'

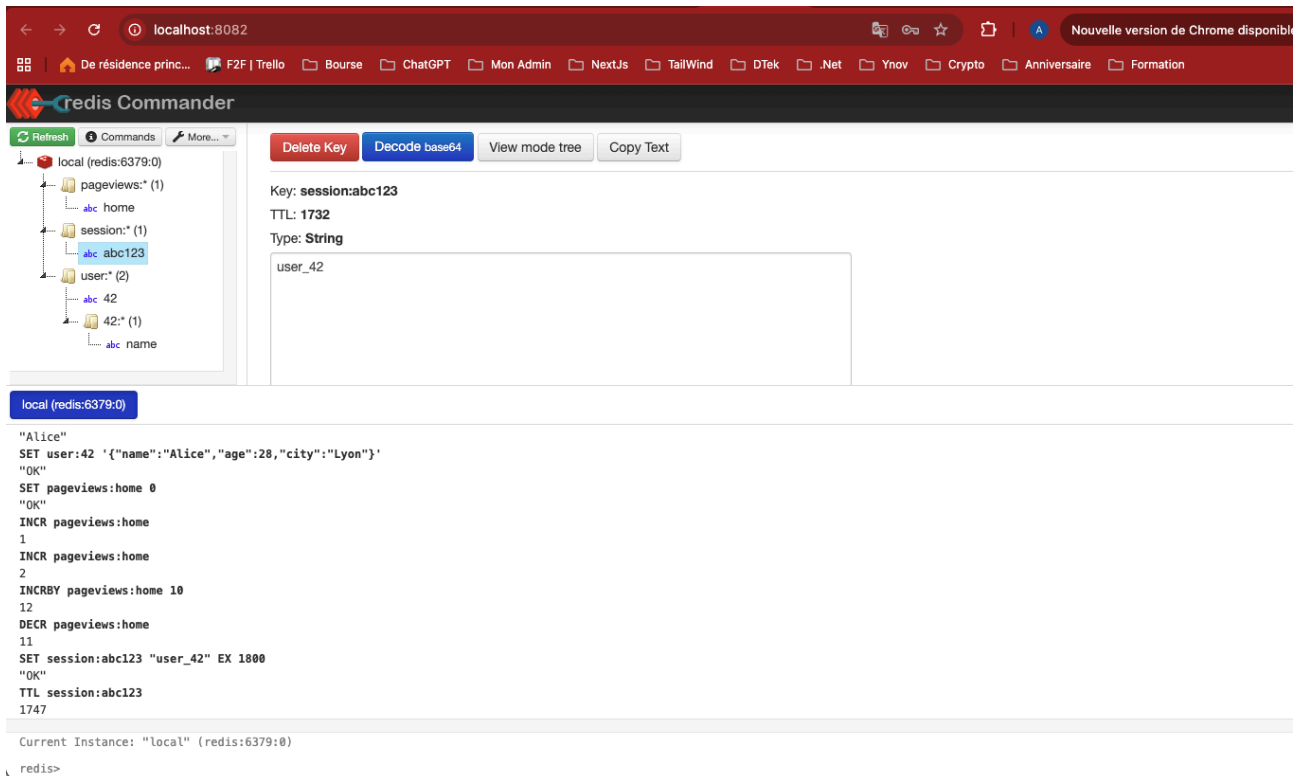
# Compteurs atomiques (parfait pour la concurrence)
SET pageviews:home 0
INCR pageviews:home    # → 1
INCR pageviews:home    # → 2
INCRBY pageviews:home 10 # → 12
DECR pageviews:home    # → 11
```


Avec expiration en une commande

SET session:abc123 "user_42" EX 1800 # expire dans 1800s (30 min)

Récupérer le TTL restant

TTL session:abc123 # → 1745 (secondes restantes)



Quand l'utiliser ? Dès que tu as besoin d'associer une clé unique à une donnée brute, que cette donnée est lue très souvent, et que tu veux pouvoir lui appliquer une date d'expiration automatique (TTL - Time To Live).

Structure 2 — Hash : objet structuré

Un Hash est une **map clé→champs→valeurs** : c'est comme un mini-objet à l'intérieur d'une clé Redis. Idéal pour modéliser un objet métier sans le sérialiser en JSON, et pour modifier un seul champ sans réécrire tout l'objet.

Création d'un objet user avec plusieurs champs

HSET user:42 name "Alice" email "alice@ex.com" age 28 city "Lyon"

Récupérer un champ

HGET user:42 name # → "Alice"

Récupérer tous les champs

HGETALL user:42

→ name=Alice, email=alice@ex.com, age=28, city=Lyon

Incrémenter un champ numérique

HINCRBY user:42 age 1 # age devient 29

Vérifier l'existence

```
HEXISTS user:42 email # → 1 (true)
```

```
# Supprimer un champ
```

```
HDEL user:42 city
```

```
# Lister tous les noms de champs
```

```
HKEYS user:42 # → [name, email, age]
```

Hash vs JSON dans String : les deux peuvent représenter un objet. **Hash** est plus efficace si vous modifiez un seul champ à la fois (pas besoin de désérialiser/sérialiser tout le JSON). **JSON dans String** est plus simple si vous lisez/écrivez toujours l'objet entier.

The screenshot shows the Redis Commander web interface in a browser. The left sidebar shows a tree view of the Redis instance 'local (redis:6379:0)' with keys 'pageviews', 'session', and 'user'. The 'user' key is selected, showing a Hash structure with fields 'name', 'email', 'age', and 'city'. The main panel displays the details for 'Key: user:42', including 'TTL: -1' and 'Type: Hash'. Below this is a table with the following data:

Field	Value
name	Alice
email	alice@ex.com
age	29
city	Lyon

At the bottom, the Redis command log shows the following commands and their results:

```
local (redis:6379:0)
HSET user:42 name "Alice" email "alice@ex.com" age 28 city "Lyon"
4
HGET user:42 name
"Alice"
HGETALL user:42
{
  "name": "Alice",
  "email": "alice@ex.com",
  "age": "28",
  "city": "Lyon"
}
HINCRBY user:42 age 1
29
HEXISTS user:42 email
1
Current Instance: "local" (redis:6379:0)
redis> HDEL user:42 city
```

Quand l'utiliser ? C'est beaucoup plus intéressant que les Strings lorsque tu as un objet avec plusieurs attributs et que ton application a besoin de modifier ou lire un seul champ à la fois (par exemple, mettre à jour uniquement l'adresse e-mail sans avoir à réécrire tout le bloc de l'utilisateur). C'est aussi très économe en mémoire.

Structure 3 — List : file ou pile ordonnée

Une List est une **séquence ordonnée d'éléments**, implémentée comme une linked list. Elle supporte les opérations en $O(1)$ aux deux extrémités, ce qui en fait la structure parfaite pour les files d'attente et les piles.

```
# Ajouter en tête (LPUSH) ou en queue (RPUSH)
```

```
LPUSH queue:emails "email1" "email2" "email3"
```

```
RPUSH queue:emails "email4"
```

```
# Récupérer la longueur
```

```
LLEN queue:emails    # → 4
```

```
# Lire sans retirer (avec indices, 0-based)
```

```
LRANGE queue:emails 0 -1 # → tous les éléments
```

```
LRANGE queue:emails 0 2 # → les 3 premiers
```

```
# Consommer en FIFO (file) ou LIFO (pile)
```

```
LPOP queue:emails    # retire et renvoie le premier (tête)
```

```
RPOP queue:emails    # retire et renvoie le dernier (queue)
```

```
# Consommer en bloquant si la liste est vide
```

```
BLPOP queue:emails 30 # attend jusqu'à 30s qu'un élément arrive
```

```
# Garder uniquement les 10 derniers (utile pour historiques)
```

```
LTRIM history:user:42 0 9
```

The screenshot shows the Redis Commander web interface. The sidebar on the left displays a tree view of the database structure, with 'queue:emails' selected. The main panel shows the details for the key 'queue:emails', including its TTL (-1) and type (List with 2 items). A table lists the elements: index 0 contains 'email2' and index 1 contains 'email1'. The bottom panel shows a log of commands executed, including 'LPUSH', 'RPUSH', 'LLEN', 'LRANGE', 'LPOP', and 'RPOP' on the 'queue:emails' key.

Quand l'utiliser ? Dès que l'ordre chronologique d'arrivée des données est crucial et que tu as besoin de construire un système de type FIFO (First In, First Out) ou LIFO (Last In, First Out).

Structure 4 — Set : ensemble unique non ordonné

Un Set est une **collection non ordonnée d'éléments uniques**. Très efficace pour les opérations ensemblistes (union, intersection, différence) et les vérifications d'appartenance en $O(1)$.

```
# Ajouter des éléments
```

```
SADD user:42:tags "premium" "fr" "verified"
```

```
SADD user:43:tags "fr" "newbie"
```

```
# Vérifier l'appartenance (en  $O(1)$  !)
```

```
SISMEMBER user:42:tags "premium" # → 1
```

```
# Lister tous les éléments
SMEMBERS user:42:tags # → ["premium", "fr", "verified"]

# Cardinalité
SCARD user:42:tags # → 3

# Opérations ensemblistes
SINTER user:42:tags user:43:tags # tags communs → ["fr"]
SUNION user:42:tags user:43:tags # tags réunis
SDIFF user:42:tags user:43:tags # de 42 sauf ceux de 43

# Élément aléatoire (parfait pour tirage au sort)
SRANDMEMBER user:42:tags 2
```

Quand l'utiliser ? Lorsque tu dois garantir qu'une donnée ne contient aucun doublon et que tu as besoin de tester instantanément si un élément fait partie d'un groupe (SISMEMBER s'exécute en temps constant $O(1)$, peu importe que l'ensemble contienne 10 ou 1 million d'éléments).

Structure 5 — Sorted Set : le secret des leaderboards

Le Sorted Set est la structure la plus puissante de Redis. C'est un ensemble d'éléments uniques, chacun associé à un score numérique, automatiquement triés par score. C'est ce qui permet de faire des leaderboards temps réel avec des millions d'utilisateurs sans broncher.

```
# Ajouter avec un score (= la position dans le classement)
ZADD leaderboard 1500 "alice"
ZADD leaderboard 1800 "bob"
ZADD leaderboard 2100 "carol"
ZADD leaderboard 1750 "dave"

# Classement par score DÉCROISSANT (top 3)
ZREVRANGE leaderboard 0 2 WITHSCORES
→ carol(2100), bob(1800), dave(1750)

# Classement croissant
ZRANGE leaderboard 0 2 WITHSCORES
→ alice(1500), dave(1750), bob(1800)

# Score d'un utilisateur précis
ZSCORE leaderboard "alice" # → 1500

# Rang d'un utilisateur (0 = premier)
ZRANK leaderboard "carol" # → 3 (4e plus petit score)
ZREVRANK leaderboard "carol" # → 0 (1er en décroissant)

# Incrémenter le score (idéal pour "alice gagne 50 points")
ZINCRBY leaderboard 50 "alice" # alice passe à 1550
```

Tous les éléments dans un intervalle de score
ZRANGEBYSCORE leaderboard 1700 1900 WITHSCORES

Cardinalité totale
ZCARD leaderboard

Performance : Sorted Set s'appuie sur une structure mixte (skip list + hash table) qui donne du $O(\log n)$ pour l'insertion et du $O(1)$ pour la recherche par membre. Un leaderboard de 10 millions de joueurs reste fluide.

Quand l'utiliser ? Dès que tu as besoin d'une liste triée en temps réel de manière dynamique, où les éléments changent de score fréquemment, et où tu as besoin d'extraire des classements ou des plages de valeurs de manière ultra-performante.

3.4. Synthèse — quelle structure pour quel besoin ?

Besoin	Structure	Commandes clés
Valeur simple, cache, compteur	String	SET, GET, INCR, EXPIRE
Objet avec champs modifiables	Hash	HSET, HGET, HGETALL, HINCRBY
File / pile / historique récent	List	LPUSH, RPUSH, LPOP, BLPOP, LTRIM
Ensemble unique (tags, follows)	Set	SADD, SMEMBERS, SINTER, SCARD
Classement / file à priorité	Sorted Set	ZADD, ZRANGE, ZINCRBY, ZRANK
Flux d'événements ordonnés	Stream (avancé)	XADD, XREAD
Cardinalité approximative	HyperLogLog	PFADD, PFCOUNT
Bitmap géographique	GeoSpatial	GEOADD, GEORADIUS

Partie 4 — Exercices de modélisation papier

4.1. La spécificité de la modélisation Redis

Modéliser pour Redis, c'est répondre à **deux questions** pour chaque besoin :

- **Quelle structure** utiliser ? (String, Hash, List, Set, Sorted Set)
- **Quelle convention de nommage** des clés ?

Contrairement à MongoDB où vous modélisez des collections et des documents, ici vous modélisez un **schéma de clés** : un fichier qui dit « pour chaque besoin, quelles clés j'utilise et avec quelle structure ».

4.2. La convention de nommage indispensable

Redis stocke toutes les clés à plat dans un espace global. **Sans convention**, votre base devient vite ingérable. La convention universelle :

```
entité:id[:sous-entité|attribut]
```

Bons exemples

user:42:name

user:42:email

user:42:profile (Hash)

user:42:notifications (List)

session:abc123 (String avec TTL)

leaderboard:weekly (Sorted Set)

station:S001:available_bikes (String avec TTL court)

Mauvais exemples (à éviter)

user42name # pas de structure

42name # ambigu

Alice # pas de typage

Règles d'or de nommage :

- **Utilisez : comme séparateur — c'est la convention universelle Redis.**
- **Préférez le singulier : user:42 plutôt que users:42.**
- **Pas d'espaces, pas d'accents, pas de caractères spéciaux.**
- **Préfixez par votre application en multi-tenant : cityflow:user:42.**
- **Évitez les clés trop longues : chaque clé consomme de la RAM, même son nom.**
- **Documentez votre schéma de clés dans un fichier REDIS_KEYS.md.**

VOTRE TOUR — Exercice 1 : Sessions utilisateurs

Vous gérez les sessions d'une application web :

- 100 000 utilisateurs simultanés possibles
- Chaque session doit stocker : userId, dernière activité, langue préférée, panier d'achat (liste de produits)
- Session expire après 30 minutes d'inactivité (renouvelée à chaque action)
- Lecture/écriture à chaque requête HTTP — performance critique

Travail demandé sur papier :

1. Quelle structure Redis utiliser pour stocker une session ? Justifiez.
2. Définissez le schéma de clés (préfixe, format).
3. Quelles sont les commandes Redis pour : créer une session, mettre à jour, prolonger sa durée, supprimer ?
4. Comment renouveler automatiquement le TTL à chaque accès ?

VOTRE TOUR — Exercice 2 : Leaderboard de jeu vidéo

Vous gérez le classement d'un jeu en ligne :

- 1 million de joueurs potentiels
- Chaque joueur a un score qui évolue dans le temps
- Besoins : top 10 global, top 10 par pays, rang d'un joueur précis, score d'un joueur
- Incrémenter le score quand un joueur gagne des points
- Réinitialiser le leaderboard hebdomadaire chaque dimanche à minuit

Travail demandé :

1. Quelle structure utiliser ? Justifiez.
2. Définissez les clés (combien de Sorted Sets en parallèle ?).
3. Écrivez les commandes pour : ajouter un joueur, incrémenter son score, lire le top 10, lire le rang d'un joueur.
4. Comment gérer le top 10 par pays sans recalculer à chaque requête ?
5. Comment réinitialiser le leaderboard hebdomadaire sans bloquer le jeu ?

VOTRE TOUR — Exercice 3 : Cache + Rate Limiting

Vous concevez un système de cache pour une API publique :

- Les réponses de l'API sont coûteuses à calculer (~500ms)
- On veut les cacher 5 minutes
- En parallèle, limiter chaque utilisateur à 100 requêtes par minute (rate limiting)
- Bloquer pendant 1 heure les utilisateurs qui dépassent

Travail demandé :

1. Définissez les clés pour le cache des réponses.
2. Définissez les clés pour le rate limiting (indice : ratelimit:user:<id>:<minute>).
3. Écrivez le flux de traitement d'une requête : que faire en premier ? Comment incrémenter le compteur ? Comment bloquer ?
4. Quel est le coût mémoire estimé pour 10 000 utilisateurs avec 100 requêtes/min ?

VOTRE TOUR — Créations multiples

Produisez votre propre jeu de données Redis riche, qui sera réutilisé pour les requêtes des parties 7 et 9. Créez au moins :

Strings (5)

- 3 compteurs : `pageviews:home`, `pageviews:about`, `pageviews:contact`
- 2 sessions avec TTL : `session:abc123`, `session:xyz789` (TTL 1800s)

Hashes (3)

- 3 profils utilisateurs distincts : `user:1:profile`, `user:2:profile`, `user:3:profile`

Lists (2)

- Notifications de 2 utilisateurs : `user:1:notifications`, `user:2:notifications`

Sets (3)

- Tags de 3 utilisateurs : `user:1:tags`, `user:2:tags`, `user:3:tags`

Sorted Sets (2)

- `leaderboard:weekly` avec au moins 10 joueurs
- `leaderboard:alltime` avec au moins 10 joueurs (les mêmes ou différents)

À la fin : vous devez avoir une quinzaine de clés visibles dans `redis-commander`.

Partie 5 — Cours : TTL, expiration et patterns avancés

5.1. Le TTL : l'arme secrète de Redis

L'une des fonctionnalités tueuses de Redis : vous pouvez **fixer une durée de vie à n'importe quelle clé**, et Redis la supprime automatiquement à expiration. C'est ce qui rend Redis parfait pour les caches, sessions, tokens temporaires.

Les commandes principales

```
# Lors du SET (en une commande)
SET session:abc123 "user_42" EX 1800      # expire dans 1800 secondes
SET session:abc123 "user_42" PX 1800000    # même chose en millisecondes
SET session:abc123 "user_42" EXAT 1735689600 # expire à un timestamp Unix précis

# Sur une clé existante
EXPIRE temp:counter 60          # expire dans 60s
PEXPIRE temp:counter 60000      # en ms
EXPIREAT temp:counter 1735689600 # à un timestamp Unix

# Vérifier le TTL restant
TTL session:abc123      # → 1745 (secondes restantes)
PTTL session:abc123     # → 1745234 (millisecondes)

# Supprimer le TTL (la clé devient permanente)
PERSIST session:abc123  # → 1 (succès)

# Codes de retour spéciaux
TTL clé_inexistante     # → -2
TTL clé_permanente      # → -1
```

5.2. Cas d'usage typiques du TTL

Cas d'usage	TTL typique	Pourquoi
Session web	30 min (1800s)	Renouvelé à chaque action
Token de vérification email	24h (86400s)	Lien d'activation
Cache de requête API	5-15 min	Tolérance aux données un peu obsolètes
Code OTP / SMS	5 min (300s)	Sécurité (court mais utilisable)
Rate limiting	60s (par minute)	Fenêtre glissante
Verrou distribué	10-60s	Évite le deadlock si crash
Notification push	Quelques minutes	Évite de l'envoyer si l'app reconnecte tard

5.3. Comment Redis gère réellement l'expiration

Question piège : que se passe-t-il exactement quand un TTL atteint zéro ? Vous pourriez imaginer un thread qui scrute toutes les clés en permanence. C'est plus subtil :

- **Expiration paresseuse (lazy)** : quand une clé expirée est accédée, elle est supprimée à ce moment-là (et GET renvoie nil).
- **Expiration active** : un processus en arrière-plan scanne aléatoirement les clés expirées, environ 10 fois par seconde.
- **Conséquence** : une clé expirée peut techniquement rester en RAM quelques secondes voire minutes avant suppression effective, si elle n'est pas accédée.

Implication pratique : si vous avez beaucoup de clés avec des TTL et qu'aucune n'est accédée, votre RAM peut monter au-delà de ce que vous attendez. La commande `DEBUG OBJECT <key>` permet d'observer le statut d'expiration.

5.4. Patterns de cache : Cache Aside vs Write Through

Pattern 1 — Cache Aside (le plus courant)

L'application gère explicitement le cache. C'est le pattern le plus utilisé.

```
# Pseudo-code applicatif
def get_user(user_id):
    # 1. Chercher dans le cache
    cached = redis.GET(f"user:{user_id}")
    if cached:
        return json.loads(cached)

    # 2. Pas en cache → aller en base SQL
    user = sql.query("SELECT * FROM users WHERE id = ?", user_id)

    # 3. Mettre en cache pour la prochaine fois
    redis.SET(f"user:{user_id}", json.dumps(user), EX=300)

    return user
```

Pattern 2 — Write Through

Chaque écriture en base est doublée d'une écriture dans Redis. Plus complexe mais garantit que le cache est toujours à jour.

Pattern 3 — Write Behind

Les écritures vont d'abord dans Redis, puis sont propagées vers la base en différé. Très performant mais risque de perte de données en cas de crash Redis.

5.5. Invalidation du cache

« **There are only two hard things in Computer Science: cache invalidation and naming things.** » — Phil Karlton

Quand une donnée change en base, comment savoir qu'elle est obsolète dans le cache ?

3 stratégies :

- **TTL court** : on accepte un délai (1-5 min). Simple, peu performant en lecture si le TTL est très court.

- **Invalidation explicite** : à chaque écriture en base, on supprime la clé Redis correspondante (DEL user:42). Plus rigoureux mais demande de la discipline dans le code.
- **Versioning** : la clé inclut un numéro de version (user:42:v3) qui change à chaque modification. Le cache n'est jamais invalidé, il est simplement contourné quand la version change.

Partie 6 — TP : commandes à produire

6.1. Objectif

Vous allez maintenant **écrire vos propres commandes** Redis pour répondre à des demandes métier. Toutes les commandes seront testées via la console de redis-commander ou via les boutons d'interface. **Notez chaque commande** dans un fichier `mes_commandes_redis.md` que vous garderez pour le projet CityFlow.

6.2. Exemples à résoudre pour démarrer

Exemple 1 : créer un compteur de visites pour la page d'accueil

Exemple 2 : stocker une session avec expiration auto de 30 min

Exemple 3 : compter le score d'un joueur dans le leaderboard

VOTRE TOUR — Commandes à produire

Pour chaque demande, **écrivez la commande Redis** dans la console, testez, et notez dans votre fichier `mes_commandes_redis.md`.

Niveau 1 — Strings et compteurs

1. Créez un compteur global de likes pour le post 100, à 0.
2. Incrémentez ce compteur 5 fois.
3. Lisez la valeur actuelle.
4. Créez un flag `maintenance:mode` à 1 avec une expiration de 60 secondes.
5. Vérifiez son TTL restant. Que se passe-t-il après 60s ? (testez en attendant)

Niveau 2 — Hashes

6. Créez un profil produit `product:101` (nom, prix, stock, catégorie).
7. Incrémentez le stock de 10 unités sans réécrire les autres champs.
8. Décrémentez le stock de 3 unités (vente).
9. Récupérez uniquement les champs nom et prix.

Niveau 3 — Lists

10. Créez une file de tâches `tasks:queue` avec 5 tâches.
11. Consommez les 2 premières tâches (FIFO).
12. Affichez ce qu'il reste dans la file.
13. Limitez la file aux 100 dernières tâches (recherchez LTRIM).

Niveau 4 — Sets

14. Créez 3 ensembles de tags pour 3 utilisateurs.
15. Trouvez les tags **communs** aux 3 utilisateurs.
16. Trouvez les tags présents chez `user:1` mais absents chez `user:2`.
17. Tirez au sort un tag parmi ceux de `user:1`.

Niveau 5 — Sorted Sets

18. Ajoutez 10 joueurs à `leaderboard:season1` avec des scores variés.
19. Affichez le top 5 par score décroissant avec leurs scores.
20. Affichez les joueurs ayant entre 1000 et 2000 points.
21. Récupérez le rang de votre joueur préféré.
22. Calculez combien de joueurs ont plus de 1500 points (ZCOUNT).

Bonus

23. Combinez deux Sorted Sets en un troisième : ZUNIONSTORE (recherchez la commande).
24. Renommez la clé leaderboard:season1 en leaderboard:archive:season1 (recherchez RENAME).

6.3. Conseils en cas de blocage

Si vous bloquez sur une commande :

1. **Cherchez dans la doc** : redis.io/commands est le meilleur point d'entrée. Tapez le nom de la commande dans la barre de recherche.
2. **Testez petit** : essayez la commande avec une clé simple avant de l'appliquer à votre vrai cas.
3. **Lisez le retour** : Redis renvoie des codes différents selon le type (int, string, array, nil). Comprenez ce qui revient.
4. **Cheat sheet en annexe** à la fin de ce document.

Partie 7 — Cours : persistance, Pub/Sub et sécurité

7.1. Persistance : RDB vs AOF

Redis stocke tout en RAM, mais la RAM est volatile : un crash ou un redémarrage effacerait tout. Pour conserver les données, Redis offre **deux mécanismes** de persistance sur disque, combinables ou utilisables séparément.

RDB — Snapshots périodiques

RDB crée des **snapshots binaires** de la base à intervalles réguliers. Le snapshot est un fichier compact (souvent compressé) qui contient l'état complet de Redis à un instant donné.

```
# Dans le fichier redis.conf
save 900 1    # snapshot si 1 modif en 900s (15 min)
save 300 10   # snapshot si 10 modifs en 300s
save 60 10000 # snapshot si 10000 modifs en 60s

# Depuis la console
CONFIG SET save "900 1"
CONFIG SET save "60 10000"

dbfilename dump.rdb
dir /data

# Sauvegarder manuellement
SAVE      # synchrone, bloque Redis (à éviter)
BGSAVE    # en arrière-plan (recommandé)
```

AOF — Append-Only File

AOF enregistre **chaque commande d'écriture** dans un fichier log. Au redémarrage, Redis rejoue toutes les commandes pour reconstruire l'état. C'est plus durable mais plus volumineux et plus lent à charger.

```
# Activation
appendonly yes
appendfilename "appendonly.aof"

# Stratégies de synchronisation (fsync)
appendfsync always  # fsync à chaque commande — durable mais lent
appendfsync everysec # toutes les secondes — compromis recommandé
appendfsync no      # géré par l'OS — rapide mais risqué
```

Que choisir selon le contexte ?

Contexte	Recommandation
Cache pur (perte acceptable)	Pas de persistance, ou RDB seul
Sessions / tokens	RDB seul (perte de quelques minutes OK)
Source de vérité	AOF + RDB (sécurité maximale)

Source de vérité critique	AOF everysec + réplication
Volumétrie immense	RDB pour rapidité de chargement

Pour ce TP, votre docker-compose.yml a --appendonly yes, donc vous avez AOF activé. Vos clés survivent aux redémarrages du conteneur. Testez : créez quelques clés, faites docker compose restart redis, vérifiez que les clés sont toujours là.

7.2. Pub/Sub : messagerie temps réel

Redis intègre un mécanisme de **publish/subscribe** qui permet à des clients de diffuser des messages sur des « canaux » et à d'autres clients de s'y abonner. Idéal pour les notifications en temps réel, chat, événements applicatifs.

```
# Terminal A — s'abonner
SUBSCRIBE notifications:user:42

# Terminal B — publier
PUBLISH notifications:user:42 "Nouveau message"

# Souscription avec pattern
PSUBSCRIBE notifications:*
```

Limite importante de Pub/Sub : les messages **ne sont pas persistés**. Si aucun subscriber n'écoute au moment du PUBLISH, le message est perdu. C'est du « fire and forget ». Pour de la fiabilité, utilisez les **Streams** (XADD/XREAD) introduits en Redis 5.

7.3. Sécurité Redis

Le problème historique

Pendant longtemps, Redis n'avait **aucune authentification par défaut** et écoutait sur toutes les interfaces réseau. Conséquence : des milliers d'instances exposées sur Internet se sont fait infecter. En 2020, le malware **H2Miner** a infecté plus de 18 000 instances Redis ouvertes pour miner de la cryptomonnaie. **Soyez vigilants.**

Les 8 commandements de la sécurité Redis

- **Ne JAMAIS exposer Redis sur Internet sans firewall.** Le port 6379 est massivement scanné.
- **Toujours activer un mot de passe** (ou ACL Redis 6+) — même en dev local.
- **Utiliser `bind`** dans redis.conf pour restreindre les interfaces d'écoute.
- **Désactiver les commandes dangereuses** : FLUSHDB, FLUSHALL, KEYS, CONFIG via rename-command.
- **Activer TLS** pour le chiffrement en transit (Redis 6+).
- **Utiliser des ACL granulaires** : pas de compte applicatif avec accès admin.
- **Monitorer** : RAM, hit rate cache, latence p99, slow log.
- **Sauvegarder le RDB régulièrement** hors du serveur (S3, NAS).

Le système ACL (Redis 6+)

```
# Avant Redis 6 : un mot de passe global
requirepass MonMotDePasseFort2025
AUTH MonMotDePasseFort2025
```

```
# Depuis Redis 6 : utilisateurs avec rôles (ACL)
ACL SETUSER cityflow_app on >AppPwd2025 ~cityflow:* +@read +@write +@hash
```

```
# Décodage :
```

```
# on          → utilisateur actif
```

```
# >AppPwd2025 → mot de passe
```

```
# ~cityflow:* → accès aux clés correspondant à ce pattern
```

```
# +@read +@write → autorise lecture et écriture
```

```
# +@hash       → autorise les commandes Hash
```

```
ACL LIST    # lister tous les users et leurs permissions
```

```
ACL WHOAMI  # voir mon user actuel
```

Dans votre stack TP, Redis est protégé par `--requirepass cityflow2025` (mot de passe simple). En production, vous utiliseriez ACL avec des comptes spécifiques par application.

Partie 8 — TP : leaderboard et cache pratiques

8.1. Objectif

Vous allez implémenter deux cas d'usage classiques de Redis : un **leaderboard avec réinitialisation hebdomadaire** et un **cache de requêtes API** avec invalidation. Ces deux patterns reviennent dans 80% des projets utilisant Redis.

8.2. TP 1 — Leaderboard hebdomadaire CityFlow

Vous gérez le classement hebdomadaire des conducteurs les plus actifs de CityFlow. À chaque trajet terminé, le score du conducteur s'incrémente.

VOTRE TOUR — Implémenter le leaderboard

Étape 1 : peupler le leaderboard

Créez le sorted set `leaderboard:weekly:trips` avec **15 utilisateurs** ayant des scores variés (entre 5 et 100 trajets). Inspirez-vous de :

```
ZADD leaderboard:weekly:trips 75 'user:1'
```

```
ZADD leaderboard:weekly:trips 92 'user:2'
```

...etc.

Étape 2 : requêtes à produire

1. Affichez le **top 5** des conducteurs avec leurs scores.
2. Affichez les 5 utilisateurs **les moins actifs** (les pires du classement).
3. Incrémentez le score de `user:1` de 3 trajets (il vient d'en faire 3).
4. Quel est le rang de `user:1` après cette mise à jour ?
5. Combien d'utilisateurs ont **plus de 50 trajets** ?
6. Listez tous les utilisateurs ayant entre 30 et 70 trajets.
7. Calculez le **nombre total d'utilisateurs** dans le leaderboard.

Étape 3 : réinitialisation hebdomadaire

8. Archivez le leaderboard actuel : renommez `leaderboard:weekly:trips` en `leaderboard:archive:2025-W19`.
9. Créez à nouveau `leaderboard:weekly:trips` vide pour la semaine suivante.
10. Vérifiez que les deux clés existent côte à côte dans `redis-commander`.

8.3. TP 2 — Cache de disponibilités de stations

Vous implémentez le **cache temps réel des disponibilités** des stations de vélos CityFlow. Une station a un nombre de vélos disponibles qui change fréquemment. On veut un accès ultra-rapide, mais accepter des données vieilles d'au maximum **60 secondes**.

VOTRE TOUR — Cache des stations

Étape 1 : peupler le cache

Créez les caches de disponibilité pour **8 stations** de Lyon avec un TTL de 60 secondes. Suivez la convention :

```
station:S001:available_bikes → String int, TTL 60s
```

```
station:S001:available_scooters → String int, TTL 60s
```

Exemples de stations : S001 (Bellecour), S002 (Part-Dieu), S003 (Croix-Rousse), S004 (Vieux Lyon), S005 (Confluence), S006 (Villeurbanne), S007 (Vénissieux), S008 (Caluire).

Étape 2 : requêtes à produire

1. Récupérez le nombre de vélos disponibles à S001.
2. Récupérez en une seule commande les vélos disponibles à S001, S002, S003 (indice : MGET).
3. Décrémentez de 1 le nombre de vélos à S002 (quelqu'un vient de louer).
4. Incrémentez de 1 le nombre de vélos à S005 (quelqu'un vient de rendre).
5. Vérifiez le TTL restant sur une clé.

Étape 3 : observer l'expiration

6. Attendez 60 secondes (ou trichez : créez une clé avec TTL 10s pour aller plus vite).
7. Vérifiez que la clé expirée a bien disparu (EXISTS doit renvoyer 0).
8. Que se passe-t-il dans votre application si vous demandez une clé expirée ? Discutez la stratégie d'invalidation : faut-il régénérer la clé à chaque expiration ?

Étape 4 : pattern Cache Aside

9. Sur papier, écrivez le pseudo-code Python ou JavaScript du pattern Cache Aside pour récupérer les vélos disponibles d'une station :
 - Étape 1 : chercher dans Redis
 - Étape 2 : si trouvé, retourner
 - Étape 3 : sinon, requêter la base de référence (Cassandra dans CityFlow)
 - Étape 4 : mettre en cache Redis avec TTL 60s
 - Étape 5 : retourner

Partie 9 — Projet fil rouge CityFlow

9.1. Objectif

Vous avez maintenant toutes les compétences pour intégrer Redis dans votre architecture CityFlow. Cette partie est **collective** : faites-la avec votre équipe projet. Le résultat alimentera directement votre repo Git.

9.2. Rappel des user stories Redis (R1-R4)

ID	User Story
US-R1	En tant qu'utilisateur, je veux voir en temps réel combien de vélos sont disponibles à la station X.
US-R2	En tant qu'utilisateur, je veux que ma session reste active pendant 30 minutes après ma dernière action.
US-R3	En tant qu'utilisateur, je veux consulter le classement des 10 utilisateurs les plus actifs du mois.
US-R4	En tant que système, je veux limiter chaque utilisateur à 100 requêtes API par minute.

9.3. Étape 1 — Modélisation du schéma de clés

VOTRE TOUR — Modélisation collective du schéma Redis CityFlow

En équipe, sur une feuille ou un tableau, concevez le **schéma de clés** complet pour supporter les 4 user stories.

Pour chaque besoin, répondez à ces 4 questions :

1. **Quelle structure** Redis utiliser ? (String, Hash, List, Set, Sorted Set)
2. **Quelle convention de nommage** (le pattern de la clé) ?
3. **Quel TTL** appliquer ? (60s, 1800s, permanent ?)
4. **Quelles commandes principales** utiliserez-vous ?

Couvrez au moins ces besoins :

- Disponibilités de chaque station (vélos, scooters)
- Sessions utilisateurs
- Leaderboard mensuel des conducteurs
- Compteurs de rate limiting par utilisateur
- (Bonus) Notifications récentes par utilisateur
- (Bonus) Tags utilisateurs (premium, eco-friendly...)

Livrable : un tableau qui résume votre schéma

9.4. Étape 2 — Création du seed CityFlow Redis

VOTRE TOUR — Création du jeu de données

Dans redis-commander (ou via la console), créez un **jeu de données réaliste** qui démontre les 4 user stories en action.

À insérer (au minimum) :

- **15 stations** avec disponibilités (vélos + scooters), TTL 60s
- **5 sessions actives** avec données JSON, TTL 1800s
- **20 utilisateurs** dans le leaderboard mensuel avec des scores variés
- **3 utilisateurs** ayant déjà un compteur de rate limiting en cours

Méthode recommandée :

- Préparez vos commandes dans un fichier texte d'abord (réutilisable plus tard pour le seed Git)
- Copiez-collez bloc par bloc dans la console redis-commander
- Vérifiez visuellement après chaque bloc dans l'arborescence de gauche

Pensez à varier : différentes villes, différents scores, différents TTL.

9.5. Étape 3 — Implémenter les 4 user stories

VOTRE TOUR — Les requêtes des user stories

Pour chaque user story, **écrivez la séquence de commandes Redis** qui la résout. Notez-les dans queries/redis-queries.md du repo CityFlow.

US-R1 : disponibilité de la station S001

→ Une commande suffit ? Deux ? Comment gérer le cas de clé expirée (cache miss) ?

US-R2 : session de 30 min renouvelée à chaque action

→ Écrivez la séquence d'actions à exécuter à chaque requête HTTP (vérification + lecture + renouvellement du TTL).

US-R3 : top 10 utilisateurs les plus actifs du mois

→ Une seule commande, avec les scores.

US-R4 : rate limiting à 100 requêtes par minute

→ Écrivez la séquence : incrémenter le compteur, fixer le TTL si nouvelle clé, vérifier le seuil, bloquer l'utilisateur si dépassement.

9.6. Étape 4 — Documenter et commiter

Pour finaliser cette séance et alimenter votre projet CityFlow :

- Dans votre repo Git, créez docs/modelisation-redis.md avec le **tableau du schéma de clés** et les justifications
- Créez seed/redis/init.sh (script shell qui peuple Redis avec votre jeu de données) — pour l'automatisation future
- Notez les 4 user stories résolues dans queries/redis-queries.md
- Faites un commit Git : feat(redis): schéma de clés et seed initial

À ce stade, vous avez :

- Une instance Redis fonctionnelle pour CityFlow, intégrée à votre stack docker-compose
- Un schéma de clés documenté et cohérent
- Un jeu de données réaliste pour la démo
- 4 user stories implémentées
- Un commit Git versionné

C'est la moitié de votre note de projet CityFlow pour la partie Redis !

Annexe A — Cheat sheet redis-commander

A.1. Navigation

Action	Où dans redis-commander ?
Se connecter	http://localhost:8082 (student / nosql2025)
Voir toutes les clés	Panneau gauche, arborescence basée sur :
Console Redis intégrée	Bouton Console en haut
Rafraîchir la liste	Bouton Refresh
Filtrer les clés	Champ de recherche en haut (pattern glob ex: user:*)

A.2. Manipuler les clés

Action	Comment ?
Créer une clé	Bouton Add Key → choisir le type
Voir une clé	Cliquer dessus dans l'arborescence
Modifier la valeur	Cliquer sur la valeur (édition inline)
Définir/modifier le TTL	Champ TTL dans la vue de la clé
Supprimer une clé	Bouton Delete rouge (avec confirmation)
Renommer une clé	Bouton Rename ou commande RENAME en console

Annexe B — Mémo des commandes Redis

B.1. Connexion et bases

```
redis-cli -h host -p 6379 -a password # connexion
PING                                # → PONG
INFO                                # infos serveur
DBSIZE                              # nb de clés dans la DB courante
SELECT 0..15                        # sélectionner une DB
FLUSHDB                             # vide la DB courante (DANGER)
```

B.2. Clés génériques

```
EXISTS key                          # 1 si existe, 0 sinon
DEL key1 key2 ...                   # supprimer
TYPE key                            # string, hash, list, set, zset
KEYS pattern                         # ⚠ DANGER en prod
SCAN 0 MATCH user:* COUNT 100      # itération non bloquante (préférer)
RENAME oldkey newkey
EXPIRE key seconds
TTL key                             # secondes restantes
PERSIST key                         # supprime le TTL
```

B.3. Strings

```
SET key value [EX seconds] [NX|XX]
GET key
MGET key1 key2 ...
APPEND key value
STRLEN key
INCR / DECR key
INCRBY / DECRBY key amount
```

B.4. Hashes

```
HSET key field1 value1 field2 value2 ...
HGET key field
HMGET key field1 field2 ...
HGETALL key
HDEL key field1 field2 ...
HEXISTS key field
HINCRBY key field amount
HKEYS key / HVALS key
HLEN key
```

B.5. Lists

```
LPUSH key val1 val2 ...      # en tête
RPUSH key val1 val2 ...      # en queue
LPOP key / RPOP key          # retire et renvoie
BLPOP key timeout            # blocking
LRANGE key start end         # 0 à -1 = tout
LLEN key
LINDEX key index
LTRIM key start end          # garde uniquement [start,end]
LREM key count value
```

B.6. Sets

```
SADD key val1 val2 ...
SREM key val
SISMEMBER key val            # 1/0
SMEMBERS key
SCARD key
SINTER key1 key2             # intersection
SUNION key1 key2             # union
SDIFF key1 key2              # différence
SRANDMEMBER key [count]
```

B.7. Sorted Sets

```
ZADD key score1 member1 score2 member2 ...
ZSCORE key member
ZINCRBY key amount member
ZRANGE key start end [WITHSCORES]
ZREVRANGE key start end [WITHSCORES]
ZRANK key member / ZREVRANK key member
ZRANGEBYSCORE key min max [WITHSCORES]
ZCOUNT key min max
ZCARD key
ZREM key member1 member2 ...
```

Annexe C — Pour aller plus loin

D.1. Utiliser redis-cli directement

```
# Se connecter au conteneur Redis en mode shell
docker exec -it cityflow-redis redis-cli -a cityflow2025

# Une fois connecté :
KEYS *
GET pageviews:home
ZREVRANGE leaderboard:weekly:trips 0 -1 WITHSCORES
```

D.2. Connecter Redis à du code

```
// Node.js avec ioredis
const Redis = require("ioredis");
const redis = new Redis({
  host: "localhost",
  port: 6379,
  password: "cityflow2025"
});
await redis.set("key", "value", "EX", 60);

# Python avec redis-py
import redis
r = redis.Redis(host='localhost', port=6379, password='cityflow2025', decode_responses=True)
r.set('key', 'value', ex=60)

// .NET avec StackExchange.Redis
var redis = ConnectionMultiplexer.Connect("localhost:6379,password=cityflow2025");
var db = redis.GetDatabase();
db.StringSet("key", "value", TimeSpan.FromSeconds(60));
```

D.3. Ressources recommandées

- **Redis University** : university.redis.com — parcours RU101 introduction gratuit
- **Documentation officielle** : redis.io/docs et redis.io/commands
- **Try Redis** : try.redis.io — sandbox interactive en ligne
- **Redis Patterns** : le livre de Salvatore Sanfilippo, gratuit sur antirez.com

À retenir absolument de cette séance

- **Redis = clé-valeur en RAM** → latence sub-milliseconde, RAM limitée.
- **5 structures à maîtriser** : String, Hash, List, Set, Sorted Set. Le Sorted Set est le secret des leaderboards.
- **Convention de nommage** : entité:id:attribut avec : comme séparateur. **Documentez-la** dans un REDIS_KEYS.md.
- **TTL est une killer feature** : sessions, caches, tokens expirent automatiquement. Coût quasi nul.
- **Persistance configurable** : RDB (snapshots) + AOF (log). À choisir selon le besoin de durabilité.
- **Cache Aside** : pattern le plus courant. L'app gère explicitement le cache (lookup, miss, set).
- **Sécurité non négociable** : authentification + bind + ACL + désactivation des commandes dangereuses.
- **Pub/Sub vs Streams** : Pub/Sub est fire-and-forget, Streams sont durables. Choix selon le besoin.